

POWER-ROUTING ALGORITHMS FOR RECONFIGURABLE MODULAR ROBOTS

Complete dual-mode and balanced flowcharts, theoretical comparison, primary-literature survey, and reproducible simulation study



MRPR REPOSITORY | TECHNICAL STUDY
Prepared 20 July 2026 | English edition

Scope: graph-based power routing, switch-state planning/runtime modes, equal-distance load balancing, electrical-topology safety, and operating-lifetime optimization.

Contents

Contents	2
Executive summary	3
1. Scope and exact graph model	4
2. Dual-mode algorithm	5
2.1 Multi-source BFS engine	6
2.2 Multi-source Dijkstra engine	7
3. Balanced equal-distance post-processor	8
3.1 Missing balanced deployment gate	9
4. Formal properties and repository comparison	10
4.1 Correctness and robustness findings	11
5. Related power-routing and sharing methods	12
5.1 Hardware, topology safety, and resilience	13
6. Experimental comparison	14
6.1 Repository demo validation	14
6.2 6x6 balancing and topology benchmark	14
6.3 3x3 energy-horizon benchmark	15
7. Recommended next optimization method	16
Required constraints	16
Lexicographic objectives	16
7.1 Immediate repository actions	16
8. Reproducibility	17
Verification performed	17
References	18

Reading note

The diagrams use enabled/disabled terminology. The source code calls a True, conducting switch 'open', which is opposite to conventional circuit-breaker wording.

Executive summary

24-40%**LOWER LOAD SPREAD**

Balanced versus nearest across the three outage suites.

26-38%**BATTERY-JOIN RATE**

Direct balanced path unions created multi-battery components.

5-14.5%**CYCLE RATE**

Cycles appeared in direct balanced path unions.

4.055**MEAN EXACT ROUNDS**

Versus 2.790 nearest, 3.120 balanced, and 3.960 greedy lifetime.

- Dual-mode is a centralized multi-source shortest-path method. Planning searches the physical graph; runtime searches only currently enabled links.
- The showcased balanced method preserves each module's shortest-route distance while improving a count-load objective through strict one-module moves.
- Balanced returns assignments and paths, not an electrically validated switch plan. Independent per-battery paths can cross ownership domains.
- For heterogeneous demands and battery capacities, the repository's energy-aware lifetime heuristic is materially closer to the restricted exact solver than count balancing.
- The recommended successor is a safety-constrained lexicographic min-cost flow or MILP with capacity, current, voltage, source isolation, radially, loss, and switch-transition terms.

Deployment warning

Do not apply a direct union of balanced paths to hardware. The repository's own Demo 3A joins B1 and B2 after one balancing move.

1. Scope and exact graph model

The robot is an undirected graph $G=(V,E)$. Selected batteries form set B ; requested action modules form set A . Each physical edge has a switch identifier and a stored nonnegative cost. Unweighted routing uses cost 1 per switch; weighted routing uses the stored edge cost.

The implemented traversal predicate is:

$$T(e) = (\text{not } \text{respect_switch_state}) \text{ OR } \text{switch_enabled}[e]$$

Mode	Flag	Eligible graph	Purpose
Planning	respect_switch_state=False	All physical edges	Plan a desired switch configuration
Runtime	respect_switch_state=True	Only currently enabled edges	Route around outages or disabled links
Unweighted	weighted=False	Unit edge cost	Minimize traversed switch count with BFS
Weighted	weighted=True	Stored edge cost	Minimize additive cost with Dijkstra

Important terminology

The source variable `switch_open=True` means conducting/traversable. This PDF says `enabled` to avoid conventional electrical ambiguity.

The flowcharts describe the demonstrated entry points:

- Dual-mode: `recommend_switch_plan()`, which calls `nearest_battery_assignment()`.
- Balanced: `rebalanced_nearest_battery_assignment()`, the main method used by the balanced demos.
- Secondary online method: `dynamic_load_balanced_assignment()`, summarized separately because it is order-dependent and not the showcased post-processor.

Unweighted forest guarantee

Every source remains a root and every discovered non-source receives one parent. A subset of the selected root-to-action paths is therefore a battery-rooted forest.

2. Dual-mode algorithm

Figure 1a expands assignment, path reconstruction, unreachable handling, and switch-plan aggregation. The method recommends states; it does not actuate hardware.

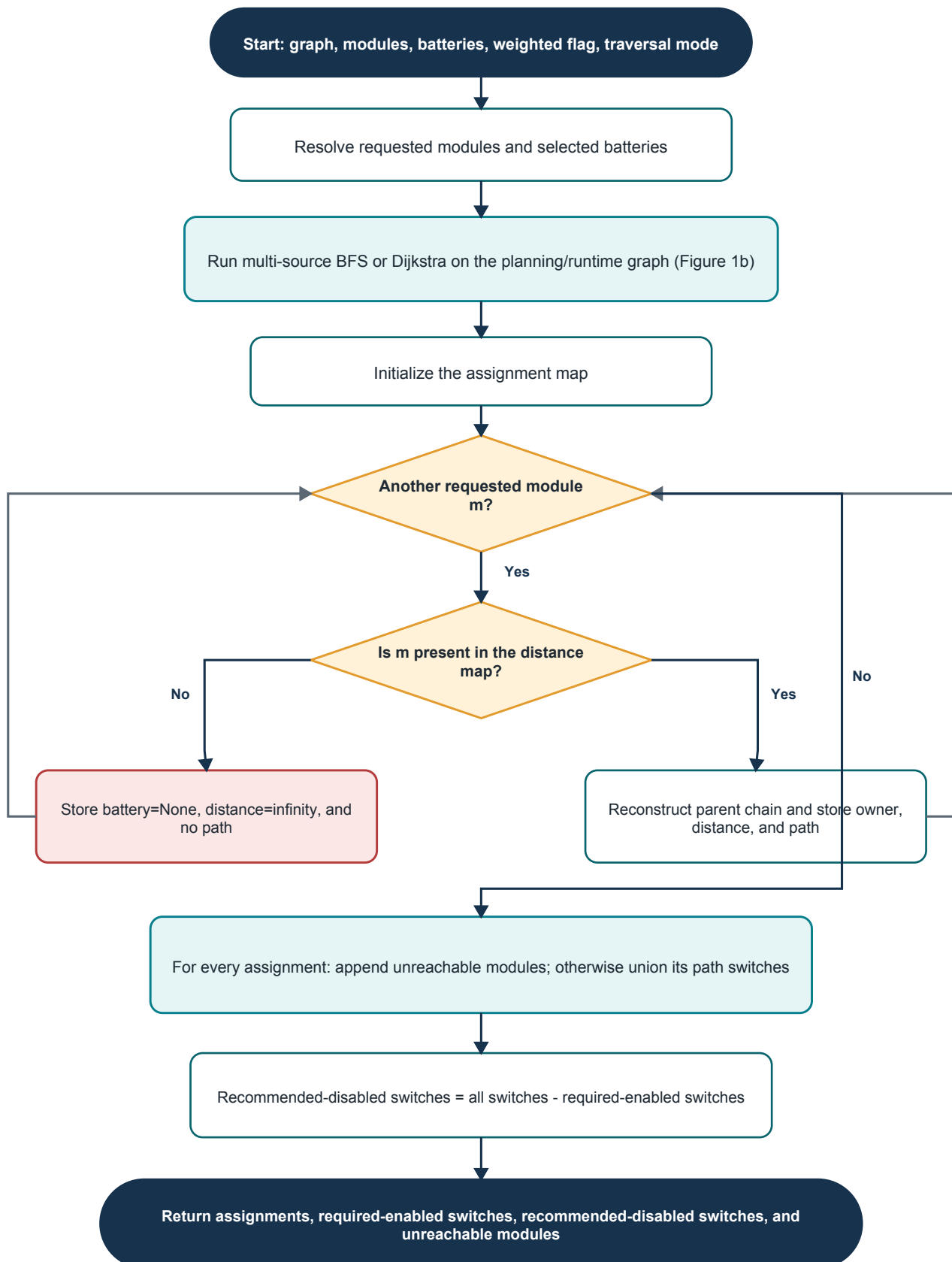


Figure 1a. Dual-mode end-to-end assignment and switch recommendation.

2.1 Multi-source BFS engine

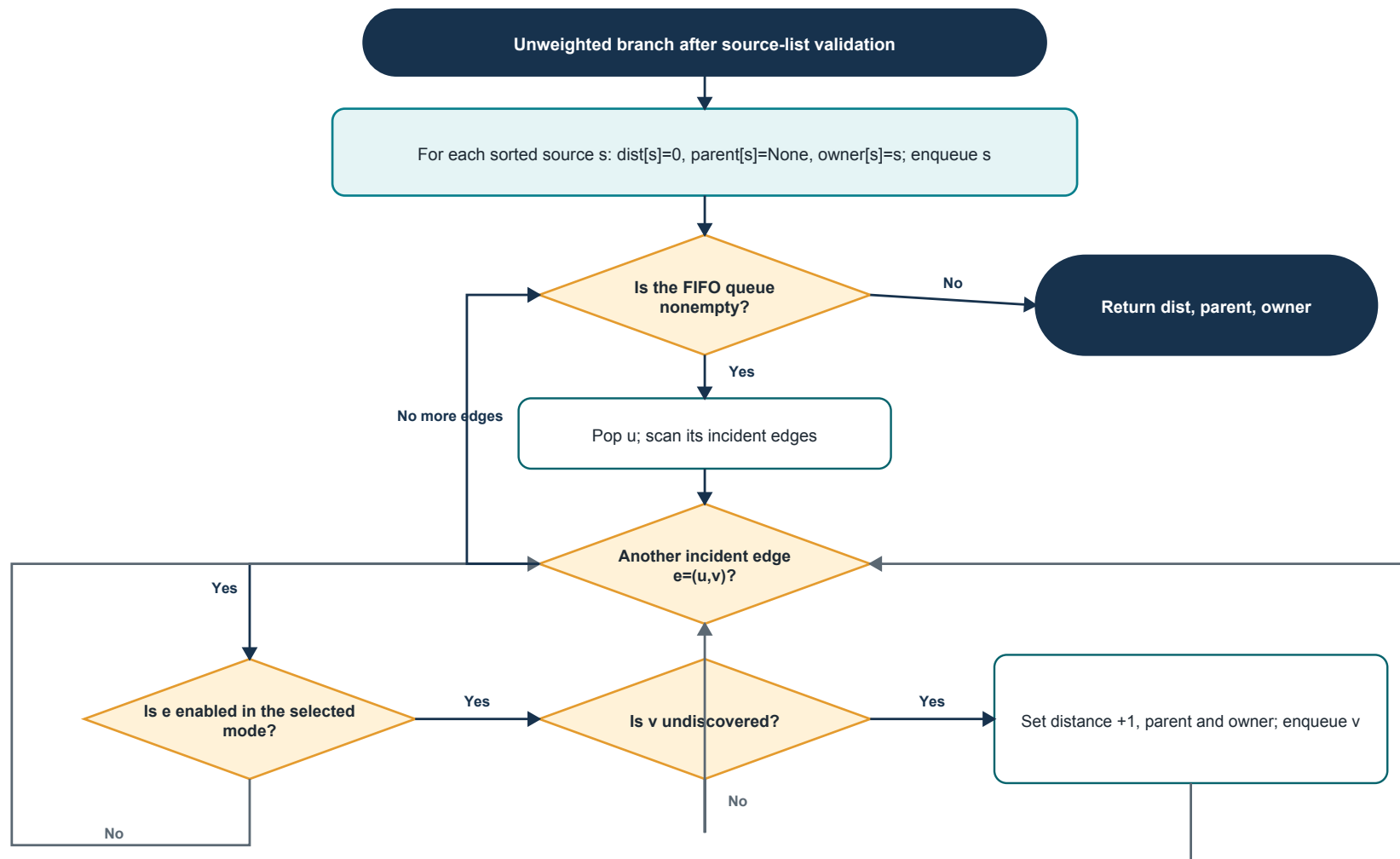


Figure 1b(i). Unweighted multi-source BFS. Source lists are validated before this branch; first discovery fixes distance, parent, and owner.

2.2 Multi-source Dijkstra engine

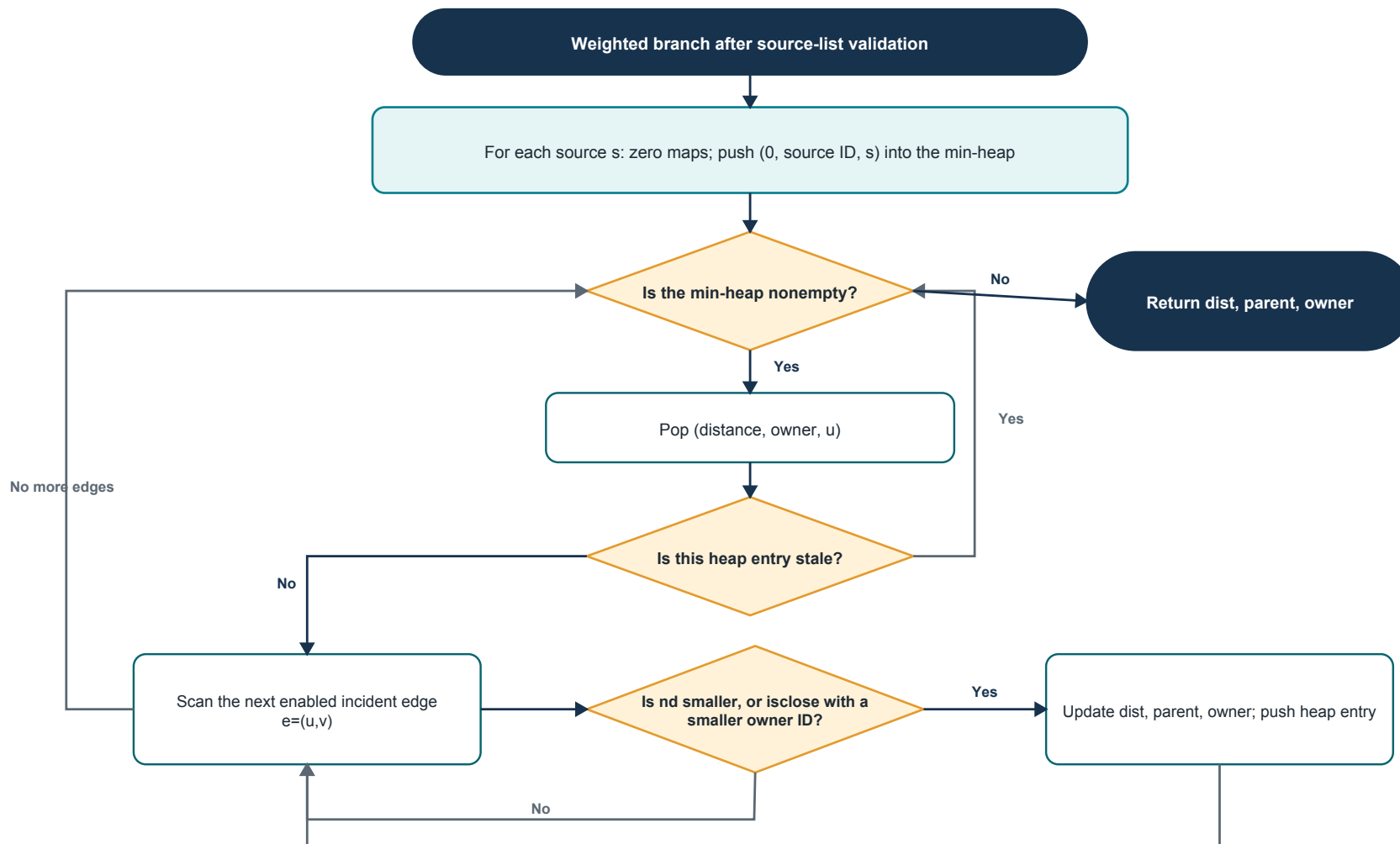


Figure 1b(ii). Weighted multi-source Dijkstra. The priority key is (distance, owner ID, node), and equal distance is tested with `math.isclose`.

3. Balanced equal-distance post-processor

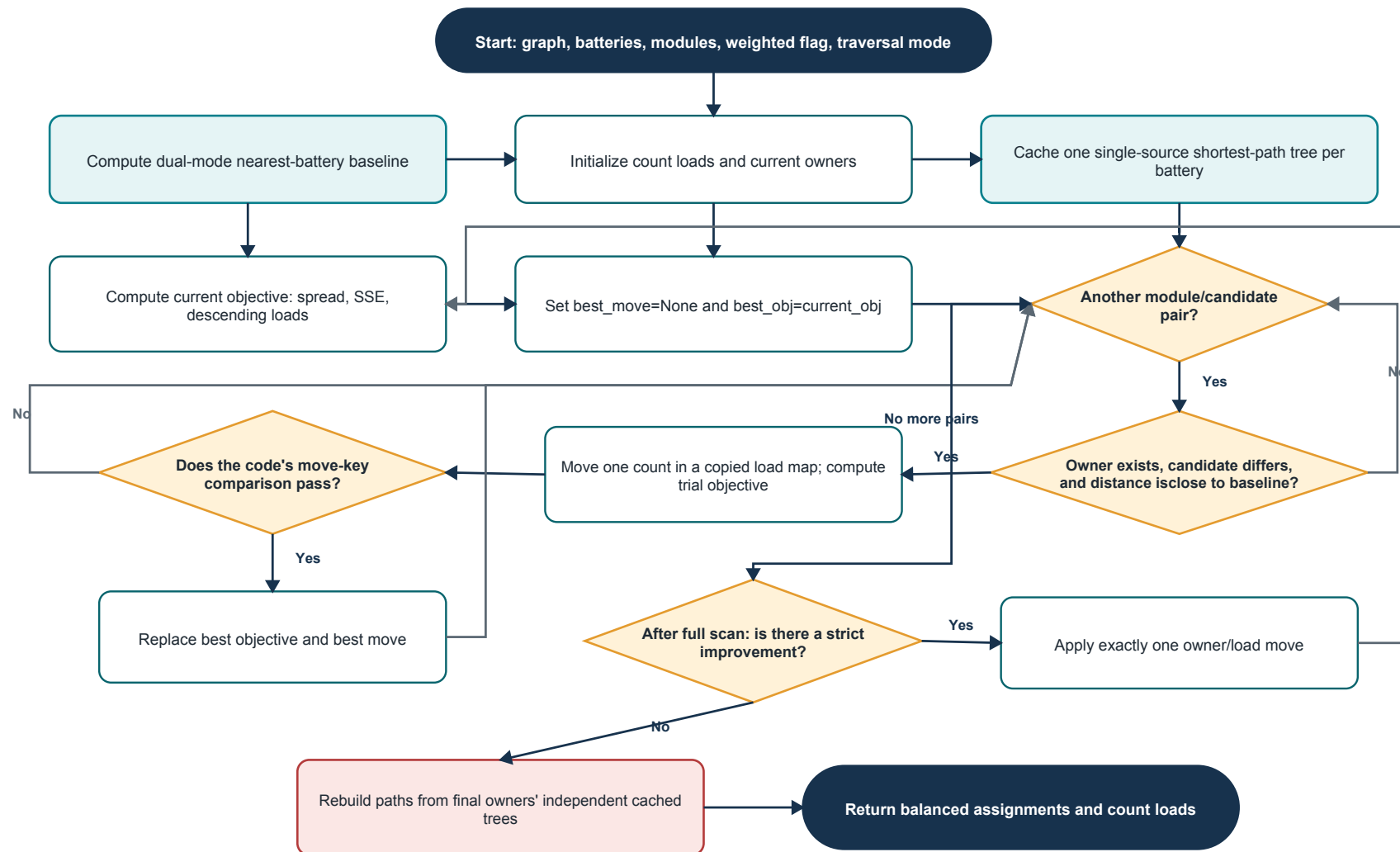


Figure 2. Code-faithful flow of `rebalanced_nearest_battery_assignment()`; every candidate move preserves baseline distance within `math.isclose` tolerance.

3.1 Missing balanced deployment gate

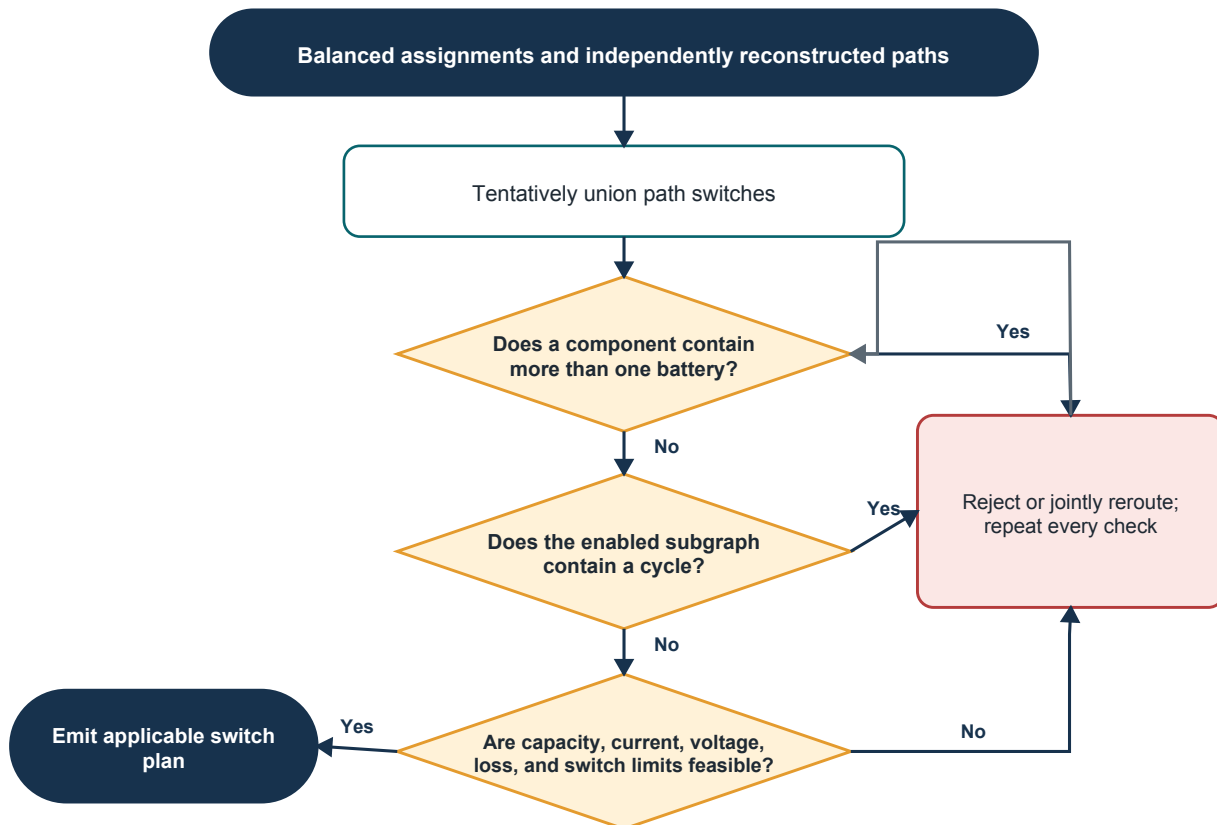


Figure 3. Mandatory safety and feasibility gate before balanced paths become a switch plan.

Current API mismatch

`power_routing_balanced.py::recommend_switch_plan()` still invokes `nearest_battery_assignment()`. It returns the unbalanced forest, not a balanced switch plan.

Weighted zero-cost edge case

The constructor accepts `cost=0`. In `B1 --0-- X --0-- B2 --1-- M`, Dijkstra reparents source `B2` to `B1` and joins both batteries. Freeze source labels or require strictly positive weighted costs.

Balanced move tie bug

The full best-move key is not retained. Equal-objective ties are won by the last scanned candidate, so explicit module or battery order can change which module moves.

The secondary `dynamic_load_balanced_assignment()` behaves differently:

- Process modules in caller order.
- For each reachable battery, rank (distance, current count, battery ID).
- Choose the minimum tuple, increment that battery count, and continue.
- The result is online and order-dependent; existing assignments can be double-counted if requested modules overlap them.

4. Formal properties and repository comparison

For action a , let $d_b(a)$ be its shortest distance from battery b . Dual-mode intends to choose the minimum (distance, battery ID). Balanced restricts each action to batteries whose distance is `math.isclose` to the baseline minimum.

$$F(L) = (\max \text{ load} - \min \text{ load}, \text{ sum of squared deviations}, \text{ descending load tuple})$$

Property	Dual-mode	Balanced post-processor
Primary objective	Minimum additive route cost for each action	Improve count-load fairness without sacrificing baseline-shortest distance
Search	One multi-source BFS or Dijkstra	Baseline search + one single-source search per battery + repeated one-move scan
Exactness	Unweighted exact; weighted ownership uses <code>math.isclose</code> tolerance	Strict single-move local optimum; not globally optimal
Unreachable actions	Explicit battery=None and infinite distance	Preserved from baseline
Tie behavior	Sorted source IDs; path tie may depend on adjacency insertion	Equal-objective move identity depends on input order because of a bug
Switch result	Forest for unweighted or strictly positive weighted costs	No balanced switch-plan API; direct union can be unsafe
Capacity / SoC / demand	Not modeled	Not modeled; load is module count only
Electrical model	Abstract additive edge cost	Same limitation
Unweighted complexity	$O(V+E)$, plus path reconstruction	Initialization $O(B(V+E))$; each iteration $O(A \cdot B^2)$ in current implementation

Local/global counterexample

Eligibility sets $m1 \rightarrow \{B0\}$, $m2 \rightarrow \{B0, B1\}$, $m3 \rightarrow \{B1, B2\}$ start at loads $(2, 1, 0)$. Each available single move only permutes that vector, so the strict loop stops; two moves reach the global $(1, 1, 1)$.

4.1 Correctness and robustness findings

Priority	Finding	Consequence / correction
Critical	Independent balanced paths can cross source domains or form cycles	Add joint switch synthesis plus source-isolation and cycle checks
Critical	Balanced switch recommender is nearest-only	Expose a separate validated balanced-plan API
High	Zero-cost weighted paths can reparent a battery source	Freeze source labels or require strictly positive weighted costs
High	No current, voltage, thermal, converter, or transient constraints	Calibrate an electrical model and enforce hardware limits
Medium	Equal-objective move identity is input-order-sensitive	Retain and compare a persistent full best-move key
Medium	math.isclose can accept a slightly longer weighted route	Set explicit numeric tolerances and document them
Medium	NaN and infinity edge costs pass validation	Require finite nonnegative costs
Medium	Explicit source type and module uniqueness are not validated	Validate selected node roles and deduplicate/reject duplicate modules
Operational	Battery health is not inferred from link state	Remove failed batteries explicitly from the source set

5. Related power-routing and sharing methods

The search used primary papers and institutional records. Direct modular-robot work is separated from transferable network-optimization baselines because the electrical abstractions and objectives differ.

Table 1. Direct power-routing and source-management methods.

Method	Objective / decision	Control and constraints	Evidence and relation to MRPR
Campbell et al. (2005) [1]	Propagate supply/ground rails with gradient, randomized, negotiation, or lattice policies	Distributed local sensing; two edge-disjoint node-covering rail subgraphs	SPICE simulation. Closest direct modular-robot routing precedent; not a shortest-path optimizer.
Wang et al. (2011) [2]	Select/share sources on a common power bus	Local hardware with overcurrent, short-circuit protection, and smooth switching	Physical modules. Strong hardware safety reference, not a graph-route optimizer.
Wang et al. e-LAN (2017) [7]	Lowest-cost source/path selection with heavy-load multi-source routing	Global link-state information; conversion/link losses and capacity screening	Case studies and simulation. Closest transferable graph-routing formulation.

Table 2. Energy-sharing and lifetime methods.

Method	Objective / decision	Control and constraints	Evidence and relation to MRPR
Raja and Scholz (2012) [3]	Threshold SoC sharing to deficient one- or two-hop neighbors	Distributed neighborhood state, directed transfer, fault isolation	Hardware measurements plus simulation. Natural SoC-aware baseline, but no global route optimum.
Chen et al. (2013) [4]	Voltage-triggered help, donor selection, five sharing circuit modes	Distributed messages; offer, bypass, receive, charge/receive	SuperBot hardware; up to 30% runtime gain in a reported imbalanced case.
Chen et al. (2016) [5]	Local consensus estimates remaining capacity and consumption to maximize first-depletion time	Distributed donor/receiver/passthrough roles; approximate conversion efficiency	Numerical and ReMod3D simulation. Stronger lifetime objective; long-path issues remain.
Chen et al. (2017) [6]	Estimate maximum operation time with output-current limits	Central and distributed allocation; capacity, demand, current-output constraints	Large random graphs and 100-SuperBot simulation. Path loss and actual route choice remain open.
Sanada et al. (2026) [11]	Longer-lived modules send power packets to shorter-lived modules	Local remaining-energy and consumption estimates	Simulation and prototype. Lifetime-aware actuation, but no compatible graph optimizer/code.

5.1 Hardware, topology safety, and resilience

Method	Objective / decision	Control and constraints	Evidence and relation to MRPR
3PAC (2022) [8]	Local plug-and-play power sharing and communication through three contacts	Distributed; empirical converter/contact circuit model	Hardware; up to 62% runtime gain in the reported simple system. Useful loss-model substrate, not a route optimizer.
FreeBOT (2022) [9]	Prune an incidence matrix to remove invalid components and maximize participation	Centralized topology; explicit short/open/cycle and polarity rules	Prototype experiments. Closest safety complement to the missing balanced deployment gate.
Mori3 resilience (2026) [10]	Share power, communication, and sensing locally to survive module deprivation	Distributed physical resource sharing and redundancy	Hardware locomotion with a resource-deprived module; official code/data available. Optimizes resilience, not route cost.

Table 3. Transferable optimization baselines.

Baseline	Mathematical contribution	What it would test in MRPR	Reproducibility
Nguyen et al. [12]	Minimum-cost flow with source cost, line capacity, flow balance, and distributed solvers	Joint source and edge capacities versus uncoupled nearest paths	High from standard algorithms and published formulation; no official code located
Baran and Wu [13]	Branch exchange over radial configurations with DistFlow loss/load approximations	Hop/count proxies versus current, resistance, voltage, and thermal limits	High for a new implementation; no official code located
Chang and Tassiulas [14]	Maximum-lifetime LP plus residual-energy-aware online routing	Dynamic energy prices versus static count balancing	High from the LP and algorithm; no official code located

6. Experimental comparison

Two deterministic, standard-library Monte Carlo studies compare repository methods under shared abstractions. They are algorithmic simulations, not physical power-electronics experiments.

6.1 Repository demo validation

Demo	Nearest loads	Balanced loads	Spread / SSE	Route distance
3A full	(5,3,4,3)	(4,4,4,3)	2 / 2.75 -> 1 / 0.75	23, unchanged
3B outages	(6,6,3)	(6,5,4)	3 / 6.0 -> 2 / 2.0	Unchanged
4A full	(2,7,4,4,3)	(2,5,4,4,5)	5 / 14.0 -> 3 / 6.0	Unchanged
4B outages	(3,6,5,6)	Unchanged	No strict move	Unchanged

Demo 3A safety counterexample

M06 moves B1->B2 along B2->M03->M02->M06 while M02 remains B1-owned along B1->M01->M02. The union joins B1 and B2 and uses 16 switches instead of 15.

6.2 6x6 balancing and topology benchmark

Configuration: 200 trials per outage rate; seed 20260720; 6x6 grid; 4 batteries; 32 requested actions; unweighted runtime routing; independent 0%, 10%, or 25% link outages.

Table 4. Route and load metrics (means over 200 trials).

Outage	Unreachable	Spread N->B	SSE N->B	Distance both	Oracle match
0%	0.00	10.745 -> 6.465	81.2400 -> 37.7000	68.105	97.0%
10%	0.11	11.535 -> 7.215	98.4925 -> 43.9425	73.355	96.5%
25%	0.95	12.450 -> 9.465	116.1200 -> 74.5300	81.310	99.5%

Table 5. Switch-union topology metrics.

Outage	Switches N->B	Multi-battery N->B	Cycle N->B
0%	32.000 -> 32.630	0.0% -> 38.0%	0.0% -> 14.5%
10%	31.890 -> 32.450	0.0% -> 38.0%	0.0% -> 10.0%
25%	31.050 -> 31.415	0.0% -> 26.0%	0.0% -> 5.0%

Balanced reduces mean spread by roughly 24-40% and load SSE by 36-55%. It matches the global equal-shortest count oracle in 96.5-99.5% of trials. The abstract fairness objective is effective, but the direct path union is not a valid switch plan.

6.3 3x3 energy-horizon benchmark

Configuration: 200 trials; seed 21160720; 2 batteries and 7 actions; capacities sampled from {30,35,...,60}; demands from {0.5,0.75,1,1.25,1.5,2}; energy = demand*(1+route distance). Lifetime and exact methods may use routes up to two switches longer than nearest.

Method	Mean rounds	Median	Mean gap to exact	Exact match
Dual nearest, static	2.790	3.0	1.265	18.5%
Balanced, static	3.120	3.0	0.935	29.0%
Greedy lifetime	3.960	4.0	0.095	91.0%
Restricted exact oracle	4.055	4.0	0.000	100.0%

Balanced beats nearest in 23.5% of paired trials, ties in 76.0%, and is worse in 0.5%. Total per-round route energy is identical; only the battery paying that energy changes.

Interpretation

Equal module count is not equal lifetime when demand and capacity differ. The greedy energy-aware method reaches the restricted exact result in 91% of trials, versus 29% for static balanced.

7. Recommended next optimization method

Use a safety-constrained lexicographic route and assignment optimizer. Precompute a small set of candidate paths from each battery to each action, excluding other selected batteries as internal nodes. Let binary variables choose a path and enable switches; add continuous flow/current variables only if split supply is permitted.

Required constraints

- Serve each critical action exactly once, or record explicit load shedding with a dominant penalty.
- A chosen path implies every switch on that path is enabled.
- Enforce battery energy/power limits and edge current limits.
- Bound voltage drop and conversion loss with a calibrated DC approximation.
- Allow at most one unsynchronized battery in each enabled connected component.
- Enforce radiality when hardware requires a forest.
- Exclude failed sources/links in runtime mode.
- Penalize switch transitions and limit chatter.

Lexicographic objectives

Priority	Objective	Reason
1	Maximize served critical load and feasible operating rounds	Safety and mission feasibility dominate every efficiency metric
2	Minimize maximum battery utilization / maximize minimum predicted lifetime	Directly addresses first-depletion operation time
3	Minimize conversion and link losses	Replaces hop count with calibrated electrical cost
4	Minimize switch changes and enabled-switch count	Reduces transient risk, wear, and unnecessary cross-links

Solver choice

A capacitated assignment or min-cost flow is sufficient for unit-demand routing without topology coupling. Source isolation, radiality, discrete switches, and voltage constraints make a MILP the clearer reference oracle.

7.1 Immediate repository actions

Order	Action
1	Add balanced switch synthesis with source-isolation, cycle, capacity, current, and voltage checks.
2	Fix the persistent full best-move key and add order-invariance tests.
3	Make battery source labels immutable in Dijkstra; validate finite costs and selected node roles.
4	Replace module count with demand- and capacity-normalized utilization.
5	Add a MILP/global oracle, then benchmark heuristic quality and runtime.
6	Calibrate edge costs from contact resistance, conversion efficiency, current, and voltage-drop data.
7	Validate switching transitions and fault isolation on hardware.

8. Reproducibility

All benchmark artifacts use only the Python standard library and the repository's existing modules.

Artifact	Path / command
Balancing and topology benchmark	python3 -B experiments/benchmark_balanced_safety.py
Energy-horizon benchmark	python3 -B experiments/benchmark_energy_horizon.py
Safety results	experiments/results/balanced_safety_summary.json and .csv
Energy results	experiments/results/energy_horizon_summary.json and .csv
English source study	POWER_ROUTING_STUDY.md

Verification performed

- All six original repository programs executed successfully.
- Both full benchmark result pairs were regenerated and byte-compared with saved JSON/CSV files.
- Benchmark assertions, JSON validation, and git diff checks passed.
- No core routing algorithm was modified for this study.

References

- [1] J. Campbell, P. Pillai, and S. C. Goldstein, 'The Robot is the Tether: Active, Adaptive Power Routing for Modular Robots With Unary Inter-robot Connectors,' IROS, 2005. <https://doi.org/10.1109/IROS.2005.1545426>
- [2] Z. Wang, F. Cordes, A. Dettmann, and R. Szczuka, 'Evaluation of a Power Management System for Heterogeneous Modules in Self-Reconfigurable Multi-Module Systems,' IROS, 2011. <https://www.dfki.de/web/forschung/projekte-publikationen/publikation/5524>
- [3] H. Q. Raja and O. Scholz, 'Dynamic Power Distribution and Energy Management in a Reconfigurable Multi-Robotic Organism,' 2012. <https://arxiv.org/abs/1207.0350>
- [4] C.-A. Chen, A. Kamimura, L. Barrios, and W.-M. Shen, 'Dynamic Power Sharing for Self-Reconfigurable Modular Robots,' TAROS, 2013. https://doi.org/10.1007/978-3-662-43645-5_1
- [5] C.-A. Chen, T. J. Collins, and W.-M. Shen, 'A Near-Optimal Dynamic Power Sharing Scheme for Self-Reconfigurable Modular Robots,' ICRA, 2016. <https://doi.org/10.1109/ICRA.2016.7487724>
- [6] C.-A. Chen, T. Collins, and W.-M. Shen, 'Maximal Operation Time Estimation for Modular and Self-Reconfigurable Robots with Output Current Constraints,' ICCAR, 2017. <https://doi.org/10.1109/ICCAR.2017.7942661>
- [7] R. Wang, J. Wu, Z. Qian, Z. Lin, and X. He, 'A Graph Theory Based Energy Routing Algorithm in Energy Local Area Network,' IEEE TII, 2017. <https://doi.org/10.1109/TII.2017.2713040>
- [8] K. Holdcroft, C. H. Belke, S. Bennani, and J. Paik, '3PAC: A Plug-and-Play System for Distributed Power Sharing and Communication in Modular Robots,' IEEE/ASME T-Mech, 2022. <https://doi.org/10.1109/TMECH.2021.3073538>
- [9] G. Liang, Y. Tu, L. Zong, J. Chen, and T. L. Lam, 'Energy Sharing Mechanism for a Freeform Robotic System - FreeBOT,' ICRA, 2022. <https://doi.org/10.1109/ICRA46639.2022.9811860>
- [10] K. Holdcroft, A. Bolotnikova, A. Jubes Monforte, and J. Paik, 'Scalable Robot Collective Resilience by Sharing Resources,' Science Robotics, 2026. <https://doi.org/10.1126/scirobotics.ady6304>
- [11] K. Sanada, N. Satoh, and H. Arai, 'Power-Sharing Using Power Packet Technology in Modular Robot and Its Evaluation,' IEICE Trans. B, 2026. <https://doi.org/10.14923/transcomj.2025JBP3012>
- [12] P. H. Nguyen, W. L. Kling, and P. F. Ribeiro, 'Agent-Based Power Routing in Active Distribution Networks,' 2010. <https://research.tue.nl/en/publications/agent-based-power-routing-in-active-distribution-networks/>
- [13] M. E. Baran and F. F. Wu, 'Network Reconfiguration in Distribution Systems for Loss Reduction and Load Balancing,' IEEE Transactions on Power Delivery, 1989. <https://doi.org/10.1109/61.25627>
- [14] J.-H. Chang and L. Tassiulas, 'Maximum Lifetime Routing in Wireless Sensor Networks,' IEEE/ACM Transactions on Networking, 2004. <https://doi.org/10.1109/TNET.2004.833122>

Literature comparison boundary

Published hardware percentage gains are not pooled with MRPR simulation results because the source/action model, electrical constraints, and objectives are not compatible. Literature evidence is contextual; repository-native experiments are the apples-to-apples comparison.